TaskPilot

Technical Documentation

The AI Agent for Your Browser — architecture reference for the web app, browser extension, AI runtime, marketplace and developer platform.

Version	2.0
Scope	Architecture (00–10)
Source	docs/architecture/
Status	Current — supersedes TaskPilot-Technical-Documentation.pdf

Contents

#	Section	Covers
00	Overview	The system in one page
01	Product Vision	What TaskPilot is for; the five products
02	Engineering Principles	The rules the code follows
03	System Architecture	Layers, service map, module ownership
04	Domain Model	Entities and relationships
05	Data Flow	How a request becomes an action
06	Execution Pipeline	Plan, step, execute, record
07	Event Architecture	Jobs, cron, notifications
08	Security Model	Trust boundaries, RLS, secrets, CORS
09	Deployment	Hosts, environments, build-time constraints
10	Performance	Caching, token budgeting, limits

00 · Overview

TaskPilot turns natural language into real browser actions. A user describes a task; the system plans it server-side and executes it in the user's own browser.

The one-paragraph version

The extension captures a compact description of the current page and sends it, with the user's instruction, to the API. The API authenticates the caller, checks their plan, and asks the runtime for a plan — heuristics first, a model only when heuristics cannot resolve the task. The resulting steps go back to the extension, which executes them against the live DOM. Results are recorded, and can be exported or replayed later as a saved agent.

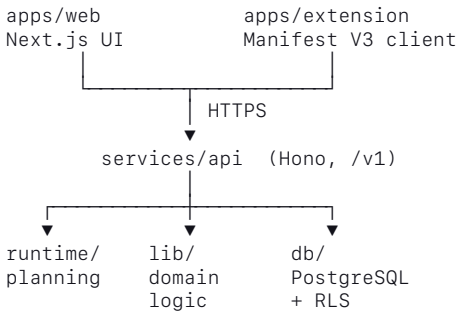
Why it is split this way

Planning is server-side so prompts, model keys and billing logic never reach a page that arbitrary JavaScript can read.

Execution is client-side so the user's existing logged-in sessions are reused. Nothing is replayed on a server that would need their credentials.

That single split explains most of the structure: `services/api` holds every secret, `apps/extension` holds every DOM capability, and `@taskpilot/shared` carries the vocabulary between them.

Shape of the system



Where to go next

- The products and their purpose — 01_PRODUCT_VISION
- The rules the code follows — 02_ENGINEERING_PRINCIPLES
- Module-by-module map — 03_SYSTEM_ARCHITECTURE

---

01 · Product Vision

> ***The AI Agent for Your Browser.***
>
> *Turn natural language into real browser actions. Fill forms, extract data,*
>
> *generate replies, automate repetitive work, and export results — all from any*
>
> *website.*

Beyond automating a single task, TaskPilot lets any workflow be packaged as an agent: reusable, shareable with a team, or sold on a marketplace.

The five products

Product	What it does	Code
Browser Extension	Executes agents inside the page	`apps/extension`
Web Application	Users, agents, billing, analytics, history	`apps/web`
AI Runtime	Plans, reasons, orchestrates	`services/api/src/runtime`
Marketplace	Publish, discover, buy, sell agents	`services/api/src/lib/marketplace.ts`
Developer Platform	SDK, typed client, API keys	`packages/sdk`, `packages/api-client`

Capabilities

What a user can do today, and the domain that owns it:

Capability	Domain
Control a website with natural language	ai, browser
Fill forms automatically	browser
Extract structured data from pages	browser
Generate AI-powered replies	ai
Upload and download files	browser
Export results to CSV, Excel, JSON	browser
Save a workflow as a reusable agent	workflows
Schedule recurring automations	workflows
Share agents with a team	auth
Publish and sell agents	marketplace
Install community agents	marketplace
Build automations without code	workflows

Long-term direction

TaskPilot aims to be the operating system for browser automation: every browser task executable through natural language, every workflow packageable as an agent, and every agent distributable — so developers can build businesses on their agents and users can automate work without writing code.

02 · Engineering Principles

These are not aspirations. Each one is observable in the code, with the place it is enforced.

1. Degrade, never crash

A missing credential disables the feature that needs it and returns `503 not_configured`. It does not take down the process, and it does not fail silently.

`GET /health` reports which subsystems are live, so "is it broken or is it unconfigured?" is answerable in one request.

> `services/api/src/server.ts` *prints the same live/disabled split at boot.*

The web app follows the rule too: `lib/server-api.ts` helpers return a fallback rather than throwing, because an unreachable API should degrade a page, not 500 it.

2. Heuristics before models

The planner resolves what it can deterministically and calls a model only when it must. Model calls cost money and latency, and a deterministic path is testable.

> `runtime/planner/heuristics.ts`, *then* `runtime/providers`.

With no API key at all, the mock provider keeps the system testable and heuristic planning still resolves common tasks.

3. Secrets live in exactly one place

`services/api` is the only package that holds credentials. It imports nothing from `apps/`, and shares only `@taskpilot/shared`.

That boundary is what makes the public/private repository split possible without a rewrite — see 09_DEPLOYMENT.

4. Plan on the server, execute on the client

Prompts and keys stay server-side. DOM access stays client-side, reusing the user's existing sessions rather than replaying credentials on a server.

5. Configure explicitly rather than derive

Behind a proxy the incoming request URL is not the public one, so values like `PUBLIC_API_URL` are configured, not inferred from the request.

The same rule applies to redirects: `signUp` passes `emailRedirectTo` rather than relying on a dashboard setting an integration can rewrite.

6. Enforce authorization at the database

Row-level security is the backstop, not the application layer. A missing `WHERE user_id = …` should not become a data leak.

> *Tested directly in* `services/api/db/tests/migrations.test.mjs`.

7. Say what is wrong, precisely

Errors name the thing that is misconfigured and what to do about it. A network failure and a rejected credential must not read the same to a user — that distinction is the difference between "try again" and "call support".

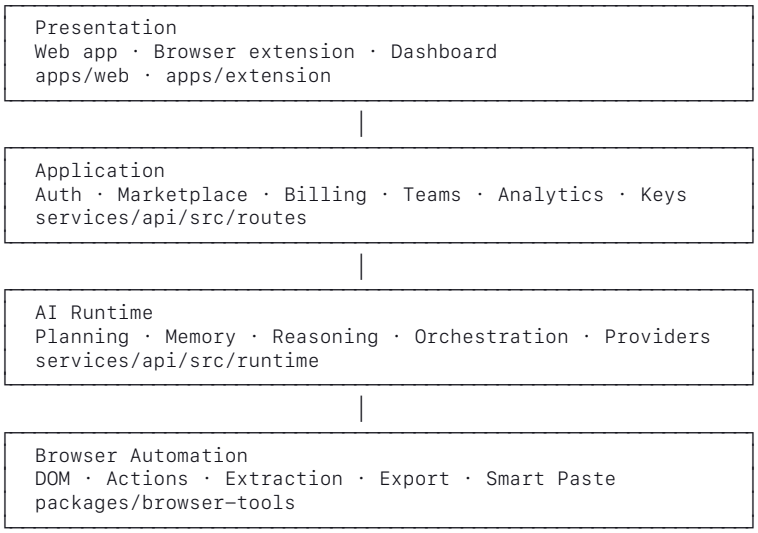
8. Tests cover the boundaries

The suite favours the seams where mistakes are expensive: RLS policies, cron arithmetic, OAuth token handling, key hashing, plan limits.

03 · System Architecture

Layers

Each layer depends only on the one beneath it. The browser automation layer holds no credentials; the application layer holds no DOM knowledge.



Service map

One Hono application (`services/api/src/app.ts`) serves everything under `/v1`. Separate concerns, one deployment — which keeps local development and the request path simple.

Route modules are **grouped, not one-file-per-route**. This is the least obvious thing about navigating `services/api`, so it is recorded explicitly: `platform.ts` holds the collaboration and distribution surfaces, `misc.ts` the operational ones.

Capability	Route	Handler	Domain logic
Authentication	`/v1/auth`	`routes/auth.ts`	—
Agent registry	`/v1/agents`	`routes/agents.ts`	`lib/agents.ts`
Runs (execution)	`/v1/runs`	`routes/runs.ts`	`lib/runs.ts`
Workflow engine	`/v1/workflows`	`routes/platform.ts`	`lib/workflows.ts`, `lib/cron.ts`
Marketplace	`/v1/marketplace`	`routes/platform.ts`	`lib/marketplace.ts`
Notifications	`/v1/notifications`	`routes/platform.ts`	`notify_user` RPC, `lib/rows.ts`
Teams	`/v1/teams`	`routes/platform.ts`	row-level security in `db/`
API keys	`/v1/keys`	`routes/platform.ts`	`lib/keys.ts`
Billing	`/v1/billing`	`routes/misc.ts`	`lib/billing.ts`
Exports	`/v1/exports`	`routes/misc.ts`	`packages/browser-tools/src/export`

Capability	Route	Handler	Domain logic
Analytics	`/v1/analytics`	`routes/misc.ts`	—
Queue workers	`/v1/jobs/worker`	`routes/misc.ts`	`lib/worker.ts`, `worker-loop.mjs`
AI entry point	`/v1/ai`	`routes/misc.ts`	`runtime/`
Integrations (OAuth)	`/v1/integrations`	`routes/integrations.ts`	`lib/oauth`, `lib/crypto.ts`

`GET /v1` returns a discovery document describing this surface at runtime.

Runtime internals

Concern	Module
Planning	`runtime/planner` — heuristics first, model second
Reasoning	`runtime/reasoner`
Memory	`runtime/memory`
Token budgeting	`runtime/optimizer/token-optimizer.ts`
Semantic cache	`runtime/cache/semantic-cache.ts`
Providers	`runtime/providers` — Anthropic, OpenAI, mock

Providers are interchangeable behind `runtime/providers/types.ts`.

Shared packages

Package	Responsibility
`packages/shared`	Domain types, manifest validation, capability catalogue
`packages/browser-tools`	Element resolver, action executor, extraction, exports
`packages/api-client`	Typed REST transport
`packages/sdk`	Author, publish and run agents

`@taskpilot/shared` is the only package both sides of the trust boundary import — it carries vocabulary, never behaviour that needs a secret.

---

04 · Domain Model

Schema lives in `services/api/db/schema.sql`, applied through seven ordered migrations in `services/api/db/migrations/`. Full reference: database/.

Entity groups

Identity and access

Table	Holds
`profiles`	User record, plan
`user_settings`, `notification_preferences`	Per-user preferences

Table	Holds
`teams`, `team_members`, `team_invites`	Collaboration and membership
`api_keys`, `api_key_usage`	Programmatic access, hashed keys
`anonymous_sessions`	Pre-signup usage

Agents

Table	Holds
`agent_versions`	Immutable manifest versions
`agent_manifests`	Manifest bodies
`agent_installs`	Which user installed what
`agent_shares`	Team-level sharing

An agent is versioned rather than mutated, so a run always references the exact manifest that produced it.

Execution

Table	Holds
`agent_runs`	One run of an agent
`agent_run_steps`	Individual planned/executed steps
`workflows`, `workflow_runs`	Saved automations and their history
`job_queue`	Deferred and scheduled work
`stored_files`	Uploads and generated artifacts

`agent_runs` is also the quota unit: the free-tier limit counts rows here since the start of the calendar month.

Marketplace

Table	Holds
`marketplace_agents`	Listings
`agent_purchases`	Purchase records and fee split
`agent_reviews`	Ratings and reviews
`referrals`	Referral attribution

AI

Table	Holds
`ai_requests`	Model call records
`response_cache`	Semantic cache entries
`saved_prompts`	Reusable user prompts

Commerce and telemetry

Table	Holds
`subscriptions`, `billing_events`	Stripe state and event log

Table	Holds
`usage_periods`	Quota accounting windows
`analytics_events`, `productivity_metrics`	Usage and outcome metrics
`notifications`	User-facing messages
`integrations`, `oauth_states`	Third-party connections, tokens encrypted at rest

Rules that hold across the model

Ownership is explicit. Rows carry the owning user or team, and row-level security enforces it at the database rather than trusting the query.

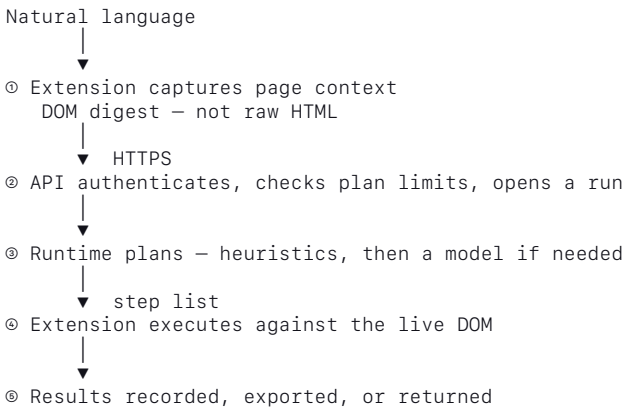
Credentials are never stored in plaintext. OAuth tokens are encrypted (`lib/crypto.ts`); API keys are stored hashed (`lib/keys.ts`).

History is append-only where it matters. Runs, steps, purchases and billing events are records of what happened, not mutable current state.

---

05 · Data Flow

How an instruction becomes an action, and what crosses each boundary.



What crosses the wire

Extension → API. The instruction, plus a *digest* of the page: candidate elements and their roles, not the full document. Sending less is both a privacy property and a token-cost property.

API → Extension. A list of steps to perform. No credentials, no prompts, no model output beyond what the step needs.

The vocabulary for both directions lives in `@taskpilot/shared`, which is why that package is the only one imported across the trust boundary.

What never crosses

Stays server-side	Stays client-side
Model API keys	The user's cookies and sessions
System prompts	Full page contents
Billing state	Local DOM handles
Service-role database access	—

Where data comes to rest

Data	Destination
Run and step records	`agent_runs`, `agent_run_steps`
Model call metadata	`ai_requests`
Cached responses	`response_cache`
Files produced	`stored_files`
Usage counters	`usage_periods`, `analytics_events`

Failure behaviour

Each stage fails distinctly, so the user sees a cause rather than a generic error:

Stage	Failure	Result
②	Not signed in	`401`
②	Over plan limit	`429` with the limit named
②	Subsystem unconfigured	`503 not_configured`
③	No model configured	Heuristic plan, or a clear message
④	Element not found	Step fails, run records why

The distinction between "unreachable" and "rejected" is deliberate — see 02_ENGINEERING_PRINCIPLES.

---

06 · Execution Pipeline

The path a single run takes, and the module responsible at each stage.

`request — authorize — admit — plan — execute — record`

Stage	Does	Module
Authorize	Identify caller — session or API key	`lib/keys.ts`, `routes/auth.ts`
Admit	Check plan limits before spending anything	`lib/runs.ts` · `assertWithinPlanLimits`
Plan	Produce ordered steps	`runtime/planner`
Execute	Perform steps against the DOM	`packages/browser-tools`
Record	Persist run, steps, usage	`lib/runs.ts`

Admission happens **before** planning. Quota is checked ahead of any model call so an over-limit user costs nothing.

Planning

The planner tries in order:

1. **Heuristics** — deterministic resolution for recognisable tasks (`runtime/planner/heuristics.ts`). Free, fast, testable.
2. **Semantic cache** — a previously computed plan for an equivalent request

(`runtime/cache/semantic-cache.ts`).

3. **Model** — a provider call, budgeted by `runtime/optimizer/token-optimizer.ts`.

Each tier only runs if the one before it did not resolve the task. With no provider configured, tier 3 falls back to the mock provider and tiers 1–2 still work.

Execution

Steps run in the extension. For each step the browser layer resolves a target element, performs the action, and reports the outcome:

Concern	Module
Finding the element	`browser-tools/src/dom`
Performing the action	`browser-tools/src/actions`
Reading data out	`browser-tools/src/extract`
Writing files out	`browser-tools/src/export`
Form autofill	`browser-tools/src/smart-paste.ts`

A step that cannot resolve its target fails that step and records the reason; it does not abort the whole run silently.

Recording

Every run writes an `agent_runs` row and one `agent_run_steps` row per step. This is what makes runs replayable, auditable, and countable for billing — the same table is the quota unit described in 04_DOMAIN_MODEL.

Deferred execution

Scheduled and background work takes the same pipeline, entered from the queue instead of a request — see 07_EVENT_ARCHITECTURE.

---

07 · Event Architecture

TaskPilot defers work through a database-backed queue rather than a message broker. One less moving part, and the queue is transactional with the data it refers to.

Queue

Piece	Where
Queue table	`job_queue`
Claim and run a batch	`lib/worker.ts`
Trigger endpoint	`POST /v1/jobs/worker` (`routes/misc.ts`)
Local ticker	`services/api/worker-loop.mjs`

The endpoint is the unit of work: something calls it, it drains a batch. That "something" can be the bundled ticker in development or any scheduler in production.

Concurrency

A batch is claimed with `FOR UPDATE SKIP LOCKED` (`db/migrations/006_platform.sql`), so several workers can poll the same table at once without colliding or double-running a job.

Situation	Behaviour
Job fails	Retries on exponential backoff via `run_after`
Retries exhausted	Parked as `dead` — not retried forever
Worker crashes mid-job	Requeued after 15 minutes (`requeue_stalled_jobs`)

Statuses are a Postgres enum: `queued`, `processing`, `succeeded`, `failed`, `dead`. The stalled-job requeue is what makes a crashed worker recoverable without manual intervention.

Why an endpoint rather than an in-process loop

The API can scale horizontally without several instances racing on the same jobs, and the trigger is observable — it either returned or it did not.

It refuses to run without `WORKER_SECRET`, so scheduled work stays off until deliberately enabled rather than running unauthenticated.

Scheduling

Cron expressions are parsed and advanced by `lib/cron.ts`, which computes the next run time for a workflow.

That module is deliberately over-tested (28 cases) — schedule arithmetic fails in ways that are silent, timezone-dependent, and only noticed days later. One test asserts it *throws* on an impossible expression rather than looping forever.

Notifications

Piece	Where
Emit	`notify_user` database function
Store	`notifications`
Preferences	`notification_preferences`
Read API	`/v1/notifications` (`routes/platform.ts`)

Emitting from a database function means a notification is written in the same transaction as the thing it describes — a run cannot complete without its notification, or vice versa.

Callers include `lib/runs.ts` and `lib/agents.ts`.

Analytics events

`analytics_events` and `productivity_metrics` record what happened for reporting. They are written on the normal path and read by `/v1/analytics`; nothing in the execution pipeline blocks on them.

---

08 · Security Model

Trust boundaries

Page content is **data, never instruction**. Text extracted from a page is never treated as a command to the planner.

Where secrets live

`services/api` is the only package holding credentials. It imports nothing from `apps/`, and shares only `@taskpilot/shared`.

Secret	Held by
`SUPABASE_SERVICE_ROLE_KEY`	API only — bypasses row-level security
Model provider keys	API only
Stripe secret and webhook secret	API only
`INTEGRATION_ENCRYPTION_KEY`	API only
`WORKER_SECRET`	API only

Anything named `NEXT_PUBLIC_*` is compiled into the browser bundle and is public by definition. Nothing sensitive may use that prefix.

Authorization

Two layers, and the second is the one that must not be skipped:

1. **Application** — route guards resolve the caller and their scopes.
2. **Database** — row-level security policies restrict what that caller can read or write.

RLS is the backstop: a forgotten `WHERE user_id = …` should be a bug, not a breach. `db/tests/migrations.test.mjs` asserts this directly, including that team members see their own team but not others', and that one user cannot read another user's integration.

Credentials at rest

Credential	Treatment
OAuth tokens	Encrypted, AES-256-GCM (`lib/crypto.ts`)
API keys	Stored hashed, shown once (`lib/keys.ts`)
Passwords	Never stored — delegated to Supabase Auth

The API refuses to start an OAuth flow without `INTEGRATION_ENCRYPTION_KEY` rather than storing a token in plaintext. A HubSpot refresh token does not expire on its own, so a database dump containing one is a standing breach.

Browser-facing controls

CORS (`app.ts · isAllowedOrigin`) allows `https://taskpilot.cc` and its subdomains, `http://localhost`, and extension origins. Extension origins are unpredictable per install, so the *scheme* is what is trusted — the browser guarantees only an installed extension can present it. Additional origins come from `ALLOWED_ORIGINS`.

CSP is set in `apps/web/src/middleware.ts`, with `connect-src` derived from `NEXT_PUBLIC_API_URL` so the allowlist cannot drift from the configured backend.

Rate limiting is per-route (`guard({ rateLimit })`), backed by Redis when configured and per-process memory otherwise — correct either way, shared only with Redis.

Redirects

`emailRedirectTo` is passed explicitly on signup rather than relying on the Supabase dashboard "Site URL", which platform integrations can rewrite. Supabase additionally requires the URL to be in the project's allowlist.

09 · Deployment

Component	Host	Build
`apps/web`	Vercel	`turbo build --filter=@taskpilot/web`
`services/api`	Railway	Container from `services/api/Dockerfile`
Database	Supabase (PostgreSQL)	Migrations in `services/api/db/migrations`
Cache / rate limits	Upstash Redis	Optional

The build-time trap

`NEXT_PUBLIC_*` values are inlined into the bundle at build time. Changing one in the hosting dashboard does nothing until the app is rebuilt.

This has bitten this project twice — once with a placeholder Supabase URL, once with `NEXT_PUBLIC_API_URL` defaulting to `http://localhost:4000` in production, which made every visitor's browser call *their own machine*. Both looked like network failures and neither was.

If a frontend value looks wrong in production, check the deployed bundle before checking the dashboard.

Runtime requirements

Node ≥ **22.13**, pinned to 24 via `.nvmrc` / `.node-version`.

The floor is not arbitrary: `packageManager` pins pnpm 11, which imports `node:sqlite` — absent on Node 20. An `engines.node` of `>=20` once let a host provision Node 20 and the install crashed before any application code ran.

The API container

Built from the **repository root**, not `services/api`, because the service depends on workspace packages outside its own directory.

Every workspace manifest is copied before installing: `--frozen-lockfile` validates the lockfile against all projects in `pnpm-workspace.yaml`, so omitting even an unrelated `package.json` fails with `ERR_PNPM_OUTDATED_LOCKFILE`.

> *Railway: leave **Root Directory at the repository root**. Pointing it at* `services/api` *makes the workspace packages unreachable.*

Environment variables

Required for the API to do anything useful

Variable	Purpose
`SUPABASE_URL`	Database — note: no `NEXT_PUBLIC_` prefix
`SUPABASE_SERVICE_ROLE_KEY`	Server-side database access

Required for correctness

Variable	Purpose
`PUBLIC_API_URL`	This service's public origin — OAuth redirects
`PUBLIC_APP_URL`	The web app's origin — email and Stripe redirects

Optional — each disables one feature when absent

`OPENAI_API_KEY` / `ANTHROPIC_API_KEY`, `STRIPE_*`, `HUBSPOT_*` + `INTEGRATION_ENCRYPTION_KEY`, `UPSTASH_REDIS_*`, `WORKER_SECRET`.

Frontend only (Vercel)

`NEXT_PUBLIC_API_URL`, `NEXT_PUBLIC_APP_URL`, `NEXT_PUBLIC_EXTENSION_ID`, `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`.

Do not set `PORT` — the host injects it and the server reads it.

Health

`GET /health` reports which subsystems are live. It is the Railway healthcheck path and the first thing to check when a feature returns `503`.

```
{ "status": "ok", "database": "configured", "ai": "unconfigured" }
```

Splitting the repository

The public/private split is a move, not a rewrite — see RUNNING.md.

---

10 · Performance

The expensive things here are model calls and DOM round-trips. Most of the design exists to avoid the first and minimise the second.

Avoiding model calls

Three tiers, cheapest first (see 06_EXECUTION_PIPELINE):

Tier	Cost	Module
Heuristic planning	Free	`runtime/planner/heuristics.ts`
Semantic cache	One lookup	`runtime/cache/semantic-cache.ts`
Model call	Money + latency	`runtime/providers`

A model call is the fallback, not the default.

Token budgeting

`runtime/optimizer/token-optimizer.ts` assigns a budget per capability, so an export step cannot consume the context an extraction step needs.

The extension also sends a **digest** of the page rather than its HTML — candidate elements and roles only. This is the single largest lever on token cost, and it doubles as a privacy property (05_DATA_FLOW).

Caching

Layer	Backing	Absent Redis
Semantic response cache	`response_cache`	Table still works
Rate limits	Upstash Redis	Per-process memory

Redis is optional. Without it both fall back to in-process state — correct, but not shared across instances. That is a scaling consideration, not a correctness one.

Quota accounting

`assertWithinPlanLimits` (`lib/runs.ts`) counts `agent_runs` since the start of the calendar month (UTC) and compares against `PLAN_LIMITS`. It runs **before** planning, so an over-limit request costs nothing.

Unlimited plans use `-1` and short-circuit before the count query.

Frontend

- Route-level code splitting via the Next.js App Router
- `next/font` self-hosts fonts — no render-blocking third-party request
- Scroll reveals use `IntersectionObserver` and respect `prefers-reduced-motion`

Cold start

`services/api` runs as a long-running container rather than serverless functions, because it holds WebSocket connections and a worker loop. Cold start is a deploy-time cost, not a per-request one.

The service starts without any credentials — missing ones disable features instead of blocking boot, so a misconfiguration never manifests as a slow start.